ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 32

Formularios y validación

- Un formulario en Laravel: la protección CSRF
- Validar en el controlador
- Mostrar los errores y recuperar lo escrito
- Las reglas de validación
- Mensajes en español
- Form Requests
- El CRUD de cursos
- PUT y DELETE desde un formulario
- Mensajes flash
- Subir archivos



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

Hasta ahora la academia solo **enseñaba** datos. Hoy aprende a **recibirlos**: el formulario de contacto, y crear, editar y borrar cursos desde la web, con imagen incluida.

Os acordáis de la clase 11, cuando validábamos a mano con un `if` por cada campo, y de la clase 25, cuando construimos nuestro `validar()` con reglas. Hoy lo hace Laravel, y vais a ver que se parece muchísimo a lo nuestro. Lo que nunca cambia es la regla de oro: **todo lo que llega del usuario se valida en el servidor**. Siempre.

•
👌 Cómo vamos a trabajar



1
📘 Un formulario en Laravel: la protección CSRF



2
👍 Validar en el controlador



3
🚀 Mostrar los errores y recuperar lo escrito



4
🔍 Las reglas de validación



5
😮 Mensajes en español



6
📘 Form Requests



7
👍 El CRUD de cursos



8
🤔 PUT y DELETE desde un formulario



9
👌 Mensajes flash



10
🚀 Subir archivos



11
🚀 Ejercicios



12
👋 Lo que tenéis que llevaros de hoy





CLASE 32

👌 Cómo vamos a trabajar

Copiad el contenido de `ejemplos/` encima del proyecto y ejecutad una vez:

terminal — php-paso-a-paso/clase32 — Visual Studio Code

terminal

```
1 php artisan storage:link
```

Esquema

```
1 app/Http/Requests/CursoRequest.php    ← NUEVO: las reglas del formulario de cursos
2 lang/es/validation.php                ← NUEVO: los mensajes de error en español
3 app/Http/Controllers/CursoController.php ← create, store, edit, update y destroy
4 app/Http/Controllers/PaginaController.php ← recibe el formulario de contacto
5 resources/views/cursos/               ← NUEVAS create, edit y formulario; show con botones
6 resources/views/paginas/contacto.blade.php ← con formulario
7 resources/views/layouts/app.blade.php ← muestra los mensajes flash
8 routes/web.php                        ← las 7 rutas de cursos y POST /contacto
```



⚠️ De momento **cualquiera** puede crear, editar y borrar cursos. En la clase 33 pondremos la puerta: solo los usuarios identificados.



CLASE 32 · PUNTO 1

📘 Un formulario en Laravel: la protección CSRF

Empezamos por el formulario de contacto (`contacto.blade.php`). Es HTML normal, con **una línea nueva**:

plantilla.php — php-paso-a-paso/clase32 — Visual Studio Code

plantilla.php

```blade
<form action="{{ route('contacto.enviar') }}" method="post">
    {{-- Sin este campo oculto, Laravel rechaza el formulario con un error 419 --}}
    @csrf

    <div class="campo">
        <label for="nombre">Nombre</label>
        <input type="text" id="nombre" name="nombre" value="{{ old('nombre') }}">
        ...
    </div>

    <button type="submit">Enviar</button>
</form>
```

¿Os acordáis del ataque **CSRF** de la clase 12? Una web maliciosa puede enviar un formulario a la nuestra aprovechando que el usuario tiene la sesión abierta. La defensa era un **token secreto** en la sesión que el formulario debía devolver. En la clase 25 lo hicimos con `campoCsrf()`. En Laravel:

- `@csrf` pinta `<input type="hidden" name="_token" value="...">`.
- Un middleware comprueba el token en **todas** las peticiones `POST`, `PUT`, `PATCH` y `DELETE` de `routes/web.php`.
- Si falta o no coincide, responde con un error **419** (*Page Expired*). Si alguna vez lo veis, lo primero: ¿he puesto `@csrf`?

Y la ruta, con su verbo:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
Route::get('/contacto', [PaginaController::class, 'contacto'])->name('contacto');
Route::post('/contacto', [PaginaController::class, 'enviarContacto'])
->name('contacto.enviar');
```



La forma más rápida de validar es el método `validate()` de la petición [1]:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
// POST /contacto: validación directamente en el controlador
public function enviarContacto(Request $request): RedirectResponse
{
    $datos = $request->validate([
        'nombre' => ['required', 'string', 'max:60'],
        'email' => ['required', 'email'],
        'mensaje' => ['required', 'string', 'min:10', 'max:1000'],
        'acepto' => ['accepted'],
    ]);

    // En la clase 37 enviaremos este mensaje por correo

    return redirect()
        ->route('contacto')
        ->with('exito', "Gracias, {$datos['nombre']}. Os responderemos muy pronto.");
}
```

¿Y si algo no es válido? **Esta es la parte bonita**: `validate()` lanza una excepción, y Laravel, sin que escribamos nada:

1. Guarda los errores y lo que había escrito el usuario en la sesión (como *flash*).
2. **Redirige a la página anterior**, al formulario.
3. Allí tenemos los errores en la variable `$errors`, y lo escrito, en `old()`.

Si todo es válido, `validate()` devuelve **solo los campos validados**. Es exactamente lo que hacía nuestra `ValidacionException` de la clase 25.



💡 Las reglas se pueden escribir como array (`['required', 'email']`) o como texto con barras (`'required|email'`). Nosotros usamos arrays: son más fáciles de leer y permiten meter objetos, como veréis en el punto 6.



CLASE 32 · PUNTO 3

🚀 Mostrar los errores y recuperar lo escrito

En la vista tenemos dos herramientas:

plantilla.php — php-paso-a-paso/clase32 — Visual Studio Code

plantilla.php

```blade
<div class="campo">
    <label for="email">Correo electrónico</label>
    <input type="email" id="email" name="email" value="{{ old('email') }}">
    @error('email')
        <p class="error">{{ $message }}</p>
    @enderror
</div>
```

- `old('email')` devuelve lo que el usuario escribió en el intento anterior. Así no tiene que volver a rellenar todo el formulario por un fallo. 🙏
- `@error('email') ... @enderror` solo pinta su contenido si ese campo tiene un error, y dentro tenemos el texto en `$message`.

Para un aviso general:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```blade
@if ($errors->any())
    <x-alerta tipo="error">Revisa los campos marcados en rojo.</x-alerta>
@endif
```

Y para los `select` y las casillas, las directivas `@selected` y `@checked`:

plantilla.php — php-paso-a-paso/clase32 — Visual Studio Code

plantilla.php

```blade
<option value="{{ $id }}" @selected(old('categoria_id', $curso->categoria_id) == $id)>{{
$nombre }}</option>

<input type="checkbox" name="publicado" value="1" @checked(old('publicado', $curso
->publicado))>
```

El segundo parámetro de `old()` es el valor por defecto: en el formulario de **editar**, lo que ya tenía el curso.



CLASE 32 · PUNTO 4

🔍 Las reglas de validación

Laravel trae más de cien reglas [2]. Estas son las que más vais a usar:

Regla	Comprueba que...
`required`	Viene y no está vacío
`nullable`	Puede venir vacío (entonces no se comprueba lo demás)
`string` , `integer` , `numeric` , `boolean`	Es de ese tipo
`email`	Es un correo válido
`min:3` , `max:120`	Longitud de un texto, valor de un número o tamaño de un archivo (en KB)
`between:1,500`	Está entre esos valores
`in:basico,medio,avanzado`	Es uno de esos valores
`date` , `after_or_equal:today`	Es una fecha / posterior o igual a hoy
`confirmed`	Coincide con el campo `xxx_confirmation` (contraseñas)
`accepted`	Se ha marcado la casilla (condiciones de uso)
`unique:categorias,nombre`	No existe ya ese valor en esa tabla y columna
`exists:categorias,id`	**Sí** existe en esa tabla (para los `select`)
`image` , `mimes:pdf,docx` , `max:2048`	Es una imagen / de esos tipos / pesa como mucho 2 MB



⚠️ **No os fiéis del `select` .** Que el desplegable solo ofrezca tres categorías no impide que alguien envíe `categoria_id=999` con las herramientas del navegador. Por eso la regla `exists:categorias,id` .



CLASE 32 · PUNTO 5

😮 Mensajes en español

Laravel trae los mensajes en inglés. Como en el `.env` tenemos `APP_LOCALE=es` , busca las traducciones en `lang/es/` . Os dejo preparado `lang/es/validation.php` con las reglas más habituales [3]:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
return [
    'accepted' => 'Debes aceptar :attribute.',
    'email' => 'El campo :attribute debe ser un correo electrónico válido.',
    'required' => 'El campo :attribute es obligatorio.',
    'max' => [
        'numeric' => 'El campo :attribute no puede ser mayor que :max.',
        'string' => 'El campo :attribute no puede tener más de :max caracteres.',
        'file' => 'El campo :attribute no puede pesar más de :max kilobytes.',
        ...
    ],
    ...

    // Nombres "bonitos" de los campos para todos los formularios
    'attributes' => [
        'email' => 'correo electrónico',
        'titulo' => 'título',
        'acepto' => 'la política de privacidad',
        ...
    ],
];
```

`:attribute`, `:max`, `:min`... se sustituyen por el nombre del campo y los valores de la regla. Sin la sección `attributes`, el mensaje diría «El campo email...»; con ella, «El campo correo electrónico...».



💡 Si necesitáis todas las traducciones (de validación, contraseñas, paginación...), existe el paquete de la comunidad `laravel-lang/lang`, que trae decenas de idiomas.



CLASE 32 · PUNTO 6

📘 Form Requests

El formulario de cursos tiene nueve campos y se usa dos veces (crear y editar). Meter todas las reglas en el controlador lo engordaría mucho. Para eso están los ***Form Requests***: una clase que contiene **solo** la validación de un formulario [4].

terminal — php-paso-a-paso/clase32 — Visual Studio Code

terminal

```
php artisan make:request CursoRequest
```

`app/Http/Requests/CursoRequest.php`:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
class CursoRequest extends FormRequest
{
    // ¿Puede este usuario enviar el formulario? Lo decidiremos en la clase 33
    public function authorize(): bool
    {
        return true;
    }

    // Antes de validar: una casilla sin marcar no se envía, así que la convertimos en
true/false
    protected function prepareForValidation(): void
    {
        $this->merge([
            'publicado' => $this->boolean('publicado'),
        ]);
    }

    public function rules(): array
    {
        return [
            'categoria_id' => ['required', 'exists:categorias,id'],
            'titulo' => [
                'required', 'string', 'min:3', 'max:120',
                // Único, salvo el propio curso cuando lo estamos editando
                Rule::unique('cursos', 'titulo')->ignore($this->route('curso')),
            ],
            'descripcion' => ['nullable', 'string', 'max:2000'],
            'imagen' => ['nullable', 'image', 'max:2048'],
            'horas' => ['required', 'integer', 'between:1,500'],
            'precio' => ['required', 'numeric', 'min:0', 'max:9999.99'],
            'plazas' => ['required', 'integer', 'between:1,100'],
            'fecha_inicio' => ['nullable', 'date', 'after_or_equal:today'],
            'publicado' => ['boolean'],
        ];
    }

    // Mensajes propios para algunas reglas (el resto sale de lang/es/validation.php)
    public function messages(): array
    {
        return [
            'titulo.unique' => 'Ya existe un curso con ese título.',
            'imagen.max' => 'La imagen no puede pesar más de 2 MB.',
            'fecha_inicio.after_or_equal' => 'El curso no puede empezar en el pasado.',
        ];
    }

    // Cómo se llama cada campo en los mensajes
    public function attributes(): array
    {
        return [
            'categoria_id' => 'categoría',
            'fecha_inicio' => 'fecha de inicio',
```

```
        ];
    }
}
```

Tres detalles importantes:

- `Rule::unique(...)->ignore(...)`: al **editar** un curso, su propio título ya está en la tabla. Sin `ignore()`, la validación diría que está repetido. `$this->route('curso')` es el curso de la URL.
- `prepareForValidation()`: una casilla sin marcar **no se envía**. Así nos aseguramos de que `publicado` siempre llegue como `true` o `false`.
- `messages()`: mensajes a medida con el formato `campo.regla`.

Y el controlador solo tiene que **pedirlo** como parámetro. Cuando el método se ejecuta, los datos **ya están validados**:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
public function store(CursoRequest $request): RedirectResponse
{
    $datos = $request->validated();   // solo los campos de rules()
    ...
}
```



CLASE 32 · PUNTO 7

👍 El CRUD de cursos

Activamos las siete rutas del recurso (clase 28):

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
Route::resource('cursos', CursoController::class);
```

Los formularios de **crear** y **editar** comparten los campos, así que los ponemos en una vista parcial, `cursos/formulario.blade.php`, que incluyen los dos. Para que funcione en ambos casos, el formulario de crear recibe un **curso vacío** con algunos valores por defecto:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
// GET /cursos/create
public function create(): View
{
    return view('cursos.create', [
        'curso' => new Curso(['plazas' => 20, 'publicado' => true]),
        'categorias' => Categoria::orderBy('nombre')->pluck('nombre', 'id'),
    ]);
}

// POST /cursos: si la validación falla, Laravel vuelve al formulario solo
public function store(CursoRequest $request): RedirectResponse
{
    $datos = $request->validated();

    if ($request->hasFile('imagen')) {
        // Guarda en storage/app/public/cursos con un nombre único y devuelve la ruta
        $datos['imagen'] = $request->file('imagen')->store('cursos', 'public');
    }

    $curso = Curso::create($datos);

    return redirect()
        ->route('cursos.show', $curso)
        ->with('exito', "Curso «{$curso->titulo}» creado.");
}
```

Fijaos en el patrón **POST → redirección → GET** (clase 11): después de guardar, **siempre** redirigimos. Si mostráramos la página directamente, al recargar el navegador volvería a enviar el formulario y crearíamos el curso dos veces.

`cursos/create.blade.php`:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```blade
{{-- enctype es obligatorio para poder enviar archivos --}}
<form action="{{ route('cursos.store') }}" method="post" enctype="multipart/form-data">
    @csrf

    @include('cursos.formulario')

    <button type="submit">Crear el curso</button>
    <a class="boton boton-secundario" href="{{ route('cursos.index') }}">Cancelar</a>
</form>
```



CLASE 32 · PUNTO 8

🤔 PUT y DELETE desde un formulario

Los formularios HTML solo saben hacer `GET` y `POST`. ¿Cómo llegamos a las rutas `PUT` y `DELETE` del recurso? Con el mismo truco que hicimos en el ejercicio 02 de la clase 26: un campo oculto que «disfraza» el `POST`. En Laravel es la directiva `@method` [5]:

plantilla.php — php-paso-a-paso/clase32 — Visual Studio Code

plantilla.php

```
<form action="{{ route('cursos.update', $curso) }}" method="post" enctype="multipart/form-
data">
    @csrf
    {{-- Los formularios solo saben GET y POST: así "disfrazamos" el POST de PUT --}}
    @method('PUT')

    @include('cursos.formulario')

    <button type="submit">Guardar los cambios</button>
</form>
```

Y el botón de borrar es **un formulario**, nunca un enlace (un enlace es un `GET`, y un `GET` jamás debe borrar nada):

plantilla.php — php-paso-a-paso/clase32 — Visual Studio Code

plantilla.php

```
<form action="{{ route('cursos.destroy', $curso) }}" method="post"
    onsubmit="return confirm('¿Seguro que quieres borrar este curso?')">
    @csrf
    @method('DELETE')
    <button type="submit" class="boton-peligro">🗑️ Borrar el curso</button>
</form>
```

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
// PUT /cursos/{curso}
public function update(CursoRequest $request, Curso $curso): RedirectResponse
{
    $datos = $request->validated();

    if ($request->hasFile('imagen')) {
        if ($curso->imagen !== null) {
            Storage::disk('public')->delete($curso->imagen);
        }

        $datos['imagen'] = $request->file('imagen')->store('cursos', 'public');
    }

    $curso->update($datos);

    return redirect()
        ->route('cursos.show', $curso)
        ->with('exito', 'Cambios guardados.');
}

// DELETE /cursos/{curso}: borrado lógico (la imagen se conserva por si se restaura)
public function destroy(Curso $curso): RedirectResponse
{
    $curso->delete();

    return redirect()
        ->route('cursos.index')
        ->with('exito', "Curso «{$curso->titulo}» borrado.");
}
```



CLASE 32 · PUNTO 9

👌 Mensajes flash

`->with('exito', '...')` guarda un mensaje en la sesión **solo para la siguiente petición**: es el *flash* que construimos en la clase 25. Lo mostramos en el layout para que funcione en todas las páginas (`layouts/app.blade.php`):

plantilla.php — php-paso-a-paso/clase32 — Visual Studio Code

plantilla.php

```
<main>
    {{-- Los mensajes flash que dejan los controladores con ->with() --}}
    @session('exito')
        <x-alerta tipo="exito">{{ $value }}</x-alerta>
    @endsession

    @session('error')
        <x-alerta tipo="error">{{ $value }}</x-alerta>
    @endsession

    @yield('contenido')
</main>
```

`@session('exito')` solo pinta su contenido si existe ese dato en la sesión, y lo deja en `$value`.

Las formas más comunes de redirigir:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```
return redirect()->route('cursos.index');            // a una ruta con nombre
return redirect()->route('cursos.show', $curso);     // con parámetros
return back();                                       // a la página anterior
return back()->with('error', 'No quedan plazas.');   // con mensaje flash
return back()->withInput();                          // conservando lo escrito
```



CLASE 32 · PUNTO 10

🚀 Subir archivos

En la clase 11 subíamos archivos con `move_uploaded_file()` y comprobábamos a mano el tipo y el tamaño. Ahora [6]:

1. El formulario necesita `enctype="multipart/form-data"` y un `<input type="file">`.
2. Validamos con `image` (o `mimes:...`) y `max:2048` (en KB).
3. Guardamos con `store()`, que genera un nombre único y aleatorio (adiós a los problemas con nombres repetidos o peligrosos):

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```
$ruta = $request->file('imagen')->store('cursos', 'public');
// "cursos/edbnVSqbsKrUuc1OqmC7UNDAwd8OWE4tLlvGjwYo.png"
```

El segundo parámetro es el **disco**. Laravel trabaja con discos configurados en `config/filesystems.php`:

Disco	Carpeta	¿Visible desde la web?
`local`	`storage/app/private`	No (documentos privados)
`public`	`storage/app/public`	Sí, gracias al enlace simbólico
`s3`	Amazon S3 (u otro servicio en la nube)	Según se configure

Recordad que solo `public/` es visible desde la web. Por eso, una vez por proyecto, creamos un **enlace simbólico** de `public/storage` a `storage/app/public`:

terminal — php-paso-a-paso/clase32 — Visual Studio Code

terminal

```
php artisan storage:link
```

Y para mostrar la imagen:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```
@if ($curso->imagen)
    <img src="{{ asset('storage/' . $curso->imagen) }}" alt="{{ $curso->titulo }}"
width="400">
@endif
```

Para borrar un archivo (lo hacemos al cambiar la imagen de un curso), la fachada `Storage`:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```
use Illuminate\Support\Facades\Storage;

Storage::disk('public')->delete($curso->imagen);
Storage::disk('public')->exists($ruta);
```



💡 En la base de datos guardamos **solo la ruta** del archivo (`cursos/xxxx.png`), nunca el archivo en sí. Por eso la columna `imagen` que añadimos en la clase 30 es un simple `string` .



CLASE 32 · PUNTO 11

🚀 Ejercicios

Las soluciones se copian dentro del proyecto, respetando sus carpetas.

- ☐ **01 · Gestionar las categorías.** Añadid a `CategoriaController` los métodos `index` (tabla con el número de cursos), `create`, `store` (validando en el controlador que el nombre sea obligatorio y único, y generando el `slug`) y `destroy`, que **no** debe borrar una categoría con cursos: en ese caso, vuelve atrás con un mensaje de error. Cuidado con el orden de las rutas. → `CategoriaController.php`, `vistas` y `routes/web.php`
- ☐ **02 · Matricular desde la web.** En la ficha del curso, un desplegable con los alumnos que todavía no están matriculados y un botón «Matricular» (`POST /cursos/{curso}/matriculas`). Validad que el alumno existe y que no está ya en el curso (`Rule::unique('alumno_curso')->where('curso_id', ...)`), y comprobad que quedan plazas. → `MatriculaController.php`, `CursoController.php`, `cursos/show.blade.php` y `routes/web.php`
- ☐ **03 · Una regla propia.** Cread con `php artisan make:rule NoGritar` una regla que rechace los textos escritos enteros en mayúsculas («CURSO DE PYTHON»), y aplicadla al título en `CursoRequest`. → `NoGritar.php` y `CursoRequest.php`



🏆 **Reto:** cread el formulario de alta de alumnos con un `AlumnoRequest` (nombre y apellidos obligatorios, correo único, teléfono opcional con la regla `regex:/^[0-9 ]{9,15}$/` y fecha de nacimiento anterior a hoy). Recordad que el modelo `Alumno` ya guarda el correo en minúsculas gracias a su *mutator*.



CLASE 32 · PUNTO 12

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
`@csrf`	Obligatorio en todo formulario `POST`; sin él, error 419
`$request->validate([...])`	Valida; si falla, vuelve al formulario con errores y datos
`old()` / `@error` / `$errors`	Recuperar lo escrito / mostrar el error de cada campo
`exists`, `unique`, `image` …	Más de cien reglas listas para usar
`lang/es/validation.php`	Los mensajes y los nombres de los campos en español
Form Request	Una clase con las reglas, los mensajes y la autorización de un formulario
`Rule::unique()->ignore()`	Único, salvo el propio registro al editar
`@method('PUT')` / `@method('DELETE')`	Formularios que llegan a las rutas `PUT` y `DELETE`
POST → redirección → GET	Después de guardar, siempre `redirect()`
`->with()` + `@session`	Mensajes *flash*
`store('cursos', 'public')` + `storage:link`	Subir archivos con nombre único y mostrarlos con `asset('storage/...')`

📚 Fuentes y documentación oficial

1. Laravel: Validation
2. Laravel: Available Validation Rules
3. Laravel: Localization
4. Laravel: Form Request Validation
5. Laravel: CSRF Protection
6. Laravel: File Storage